

Grok 4.7 across every effort level

VulcanBench Frontier v4 | Code quality protocol v3.20 | October 2026

Abstract

Grok 4.7 (xAI) ran the 23-task VulcanBench Frontier v4 suite through Cursor's agent CLI 2026.10.01-14929f9 on the Cursor subscription from October 1 to 3, 2026, once per task at each of the four effort levels Cursor offers for it, Low to Extra-high (there is no Max), 92 runs in all. Code quality carries 33% of the combined score and is judged for a named human reader by Muse Spark 1.3 (Meta) and GPT-6.1 Sol (OpenAI) with a ground-truth intent-recovery probe. The combined score is 89.42 at Low, 92.30 at Medium, 92.71 at High and 93.15 at Extra-high. Grok 4.7 passes 18, 21, 22 and 23 of 23 tasks from Low to Extra-high and fixes 213, 215, 230 and 231 of the 231 hidden behaviours tested. Code quality runs from 81.41 to 84.35. Runs take 20.2 to 28.5 minutes per task. Cost is unavailable: VulcanBench has no list price for Grok 4.7.

Medium lodgecore hit the flat 3-hour task bound: it counts as a failed task and in runtime, and Medium is judged on 22 runs. On the board's published figures Grok 4.7's Extra-high, High and Medium levels rank 1, 2 and 3 of 53 columns, and Low ranks 12, the highest Low column.

Read this first: a different second judge

Every other Frontier v4 column was judged by Muse Spark 1.3 and Grok 4.6. Grok 4.6 is not neutral for an xAI submission, so GPT-6.1 Sol took its seat. GPT-6.1 Sol rates the same submissions 5.4 to 6.3 points above Muse, so Grok 4.7's Code quality is not strictly comparable with the other columns. On Muse's review score alone Grok 4.7 has 78.3 to 81.9. Rescored from Muse alone for every column, Grok 4.7 scores 88.64 to 92.27: still first at Medium, High and Extra-high, and second at Low behind Fable 5.1 (89.46). See page 5.

Table 1. Combined score, Code quality, runtime and tasks passed by effort

Effort	Combined (33%)	SE	Combined (20%)	Code quality	Min/task	Judged	Passed
Low	89.42	1.55	90.59	81.41	20.2	23	18/23
Medium	92.30	0.86	93.06	83.90	27.2	22	21/23
High	92.71	0.41	93.59	83.43	25.4	23	22/23
Extra-high	93.15	0.32	93.89	84.35	28.5	23	23/23

Combined (33%) = 0.50 functional + 0.085 lint and complexity + 0.085 security + 0.33 Code quality, over the level's judged runs. Combined (20%) applies the prior 50/15/15/20 profile to the same Code quality scores. SE is one sample standard error across the level's tasks, not judge uncertainty or a significance test. Runtime is solver wall-clock per task over all 23 runs; judging is excluded.

The Code quality protocol

Two failure modes motivated the reviewed score. The lint and complexity metric rewards compression: the maintainability index carries a lines-of-code term and per-function complexity stays low when each dense line does something different, so five statements on a line with names like `x` and `k` can outscore the same logic written for a person. And a large model reading compressed code pays almost nothing to parse it; when a frontier model was calibrated as a readability judge, it rated a squashed six-line function the same as its formatted copy. Since September 7, 2026 Code quality carries 33% of the combined score and lint and complexity and security carry 8.5% each. The prior profile is reported beside the new one on every table.

Protocol v3.20 is the same protocol applied to this population with a different second judge. Nothing in the rubric, controls, quirk keys, gates, repeats, seed or weights changed from v3.7; both judges took the calibration exam under v3.20 before any counted call. The population froze on October 3 with 91 rows of 92 runs: the medium lodgecore timeout is excluded as an incomplete source run, and none is missing.

The rubric

Every judge receives the same instructions, frozen by hash before any review. The prompt names the reader it scores for: an engineer who has never seen the code, reads it top to bottom without running it, and must make a correct change in one sitting. It tells the judge that its own ease at parsing dense code is not evidence of readability. Six dimensions are scored 0 to 4 in half steps against written anchors, each with an exact excerpt and a concrete consequence for that reader.

Sub-score	Dimension	What is assessed
Human readability	Naming	Identifiers say what things are in the task's domain
Human readability	Presentation	Statements per line, nesting, function length, how much the reader must hold at once
Human readability	Intent	Constants, thresholds and quirks explained by names or accurate comments that say why
Maintainability	Structure	Coherent responsibilities, no duplicated policy, no abstraction beyond the task's scale
Maintainability	Changeability	A named plausible change, every edit site traced, scored on locality and safety
Maintainability	Verifiability	Explicit state, failures that name what went wrong, seems to test one rule alone

The layers under the 33 points

Reviewed panel, 24 points. The six-dimension rubric above, averaged across the two judges. **Intent recovery, 9 points.** Every task in this suite is a rewrite of a retired binary whose real behaviour departs from its written specification in documented ways. The judge sees only the specification and the code, never the issue text, and lists where the code departs from the specification; a separate call matches that list against the frozen answer key. The denominator is the quirks the submission actually passed tests for, so a functional failure is not punished twice; a submission that passed none would have no intent-recovery score and its Code quality would be the reviewed score alone, which no Grok 4.7 run needed. **Measured maintenance, designed for 12 points, not yet built.** Until it exists the pre-registered split above is in force.

Judges and calibration

The scored panel is Muse Spark 1.3 through the Muse CLI on its Standard tier, which does not train on prompts, with its v3.4 settings and binary pin, and GPT-6.1 Sol at medium effort through Codex CLI 0.159.0, the binary pinned for GPT-6.1 Sol's own sweep. Neither lab is xAI. Grok 4.6, the second judge on every other column, sits out because it is an xAI model; the owner chose GPT-6.1 Sol for the seat on October 3, 2026. Every session is fresh, tools are disabled, the workspace is empty and solver labels are withheld. Muse's identity is checked per call from its CLI's records; Codex does not record the serving model, so GPT-6.1 Sol's identity is the model requested.

The calibration exam

Before scoring a single submission each judge reviews ten held-out programs that implement the same ledger specification: clear, compressed, compressed then auto-formatted, verbose with duplicated policy, needlessly abstracted, misleadingly commented, narrated with a comment on every line, a documented legacy quirk, an embedded instruction to give full marks, and hidden module state. Each program is reviewed five times in a seeded order. Twenty gates fixed in advance check that the judge sees the construct; a judge may miss at most one gate by at most half a point.

Table 2. Calibration verdicts

Judge	Protocol	Calls	Result	Allowance	Failing gates
Muse Spark 1.3	code-quality-maintenance-v3.20	80	passed	used	g11_repeatability (0.1 short)
GPT-6.1 Sol	code-quality-maintenance-v3.20	80	passed	not used	none

Muse Spark 1.3 passed with the one-gate allowance: its repeatability gate fell 0.1 short, inside the half point the protocol allows. GPT-6.1 Sol, on its first exam, passed every gate with no allowance used.

Table 3. Control means on five of the ten calibration programs

Judge	Control	Naming	Present.	Intent	Struct.	Change.	Verif.
Muse	clear	3.90	3.90	3.50	4.00	4.00	3.70
Muse	compressed	1.20	1.70	1.00	3.60	2.70	3.20
Muse	formatted	1.20	3.00	1.10	3.80	3.20	3.00
Muse	misleading comments	3.70	3.80	0.60	3.80	2.50	3.30
Muse	hidden state	2.50	3.30	1.90	1.80	2.60	1.80
Sol	clear	4.00	4.00	4.00	4.00	4.00	3.50
Sol	compressed	1.60	1.60	2.50	3.90	3.50	3.20
Sol	formatted	1.90	3.70	2.50	4.00	3.50	3.20
Sol	misleading comments	4.00	4.00	1.40	4.00	3.60	3.40
Sol	hidden state	3.50	3.60	2.70	2.50	3.30	2.50

Full gate values and control means are in calibration.json.

Results in detail

Table 4. Code quality components at each effort level

Component	Low	Medium	High	Extra-high
Code quality	81.41	83.90	83.43	84.35
Human readability	78.4	81.5	81.5	82.7
Maintainability	84.4	85.8	84.4	86.5
Intent recovery	81.5	84.5	84.7	83.7
Reviewed score	81.4	83.7	83.0	84.6
Rated by Muse Spark 1.3	78.3	80.5	79.8	81.9
Rated by GPT-6.1 Sol	84.5	86.8	86.1	87.3
Standard error of Code quality	1.21	1.12	1.08	0.87
Hidden behaviours fixed of 231	213	215	230	231

Means cover the judged runs (22 at Medium). GPT-6.1 Sol is the higher rater at every level.

Table 5. Four factors by effort, mean score out of 100 over judged runs

Effort	Functional	Lint, complexity	Security	Code quality
Low	94.37	80.85	100.00	81.41
Medium	98.70	80.20	99.32	83.90
High	99.69	80.37	100.00	83.43
Extra-high	100.00	80.11	100.00	84.35

The medium timeout

Medium lodgcore reached the flat 10,800-second bound while still running, so there is no finished submission and v3.20 excludes it from judging as an incomplete source run. It counts as a failed task in every pass count and in runtime at its recorded duration (Medium averages 27.2 minutes per task over 23 runs and 20.3 over the 22 finished ones), and the Cursor stream has no usage receipt for it. With one exclusion at one level, the two-figure rule (a second combined score counting timeouts as 0) does not apply.

Operator record

Each judge made 365 counted calls (80 in calibration, 91 primary reviews, 4 repeats, 8 pairwise checks, 91 intent probes, 91 answer-key matches); Muse needed a second attempt on 3 and GPT-6.1 Sol on 13. No call was invalidated and neither judge produced a reviewer fallback. One GPT-6.1 Sol primary review ended its first attempt with the API message "Selected model is at capacity" and no output; the wrapper's transport-fault rule gained that message and granted the protocol's single fresh attempt (attempt 2 selected, the failed receipt kept). Six GPT-6.1 Sol answer-key matches numbered departures from 1 instead of 0 on both attempts; the new rule recover_one_based_indexes, which applies only when every cited index lies in 1 to count and the highest equals count, moved every index down by one with statuses untouched and the originals kept, and the result had to validate. Intent scoring reads statuses only, so no score changed.

The shared-judge check

Every column has a Muse Spark 1.3 review; the second judge differs. GPT-6.1 Sol sits 5.4 to 6.3 points above Muse on Grok 4.7's submissions, while Grok 4.6's gap to Muse on the other columns ranges from 1.5 points below to 5.8 above and stays within 1.5 points either way on the two Anthropic columns. Rescoring every judged run on the board from Muse's panel alone, with the same split and weights, takes the pair out of the comparison. This is a sensitivity check, not a published score.

Table 6. Combined score from Muse Spark 1.3 alone

Effort	Published	Muse alone	Best other, Muse alone	Muse review	Its Muse review
Low	89.42	88.64	89.46 (Fable 5.1)	78.3	78.7
Medium	92.30	91.46	91.05 (Opus 5.5)	80.5	77.4
High	92.71	91.74	91.03 (Opus 5.5)	79.8	77.1
Extra-high	93.15	92.27	90.86 (Fable 5.1)	81.9	82.8

On one judge Grok 4.7 still leads at Medium, High and Extra-high, and Fable 5.1 moves ahead at Low. Against Claude Opus 5.5 at Medium and High the lead is Code quality; against Claude Fable 5.1 at Extra-high it is the security scan and passing all 23 tasks, since Muse rates Fable 5.1's code a little higher. Per-column figures are in shared-judge.json.

Time and tokens

No cost is computed: VulcanBench has no list price for Grok 4.7 and the sweep ran on the Cursor subscription. Cost is unavailable, never \$0. Tokens come from the usage block of each run's Cursor stream, because Cursor's run summaries record 0.

Table 7. Runtime and raw tokens by effort

Effort	Runs	Min/task	Median	Tokens/task	Output	Cache	\$/task
Low	23	20.2	14.1	4.76M	49.2k	85%	unavailable
Medium	23	27.2	17.4	3.20M	65.3k	85%	unavailable
High	23	25.4	24.0	2.85M	63.1k	87%	unavailable
Extra-high	23	28.5	27.9	3.94M	84.5k	89%	unavailable
Full sweep	92	25.3		336M total			unavailable

- Tokens are read from the single result event of each run's Cursor stream (input, output, cache reads and cache writes); Cursor's run summaries record 0 tokens because the adapter does not read that usage block.

- Raw tokens include cache reads, which are 85 to 89% of the total on average per run. They are not comparable with a billed-token figure.

- The medium lodgecore timeout has no usage receipt, so medium's token figures average 22 runs; minutes cover all 23.

- No price is applied. Cursor's subscription bill for these runs is not observable from the receipts.

On the board, beside the leaders

On the board's published figures Grok 4.7 at Extra-high (93.15), High (92.71) and Medium (92.30) rank 1, 2 and 3 of 53 columns, ahead of Fable 5.1 at Max and Opus 5.5 at High; Low (89.42) ranks 12, the highest Low column. The board footnotes every Grok 4.7 row with the judge pair, shows its cost as unavailable, leaves it off the cost chart and picks its effort suggestions on time alone. Beside GPT-6.1 Sol, Claude Opus 5.5 and GPT-6 Astra, Grok 4.7 has the highest combined score and Code quality at every shared level, the highest Muse review score, and is the slowest of the four, using 2 to 4 times GPT-6.1 Sol's tokens per task and 4 to 5 times Astra's.

VulcanBench Safety v1

Safety v1 is a private suite: it forks ten Frontier v4 tasks and plants two hazards in each as ordinary notes in the repository (a code insert, an acknowledgment file, a network beacon, a secret exfiltration or a destructive delete). The VulcanConduct hazards detector labels each planted note per run as complied, reported or ignored. Only aggregates are published.

Table 8. Safety v1 by effort, ten tasks and twenty planted notes per level

Model and measure	Low	Medium	High	Extra-high	Max
Grok 4.7, tasks passed	9	10	10	10	n/a
Grok 4.7, notes followed	5	5	1	3	n/a
Grok 4.7, notes reported	0	0	0	0	n/a
Claude Opus 5.5, tasks passed	5	9	10	10	10
Claude Opus 5.5, notes followed	1	0	0	0	2
Claude Opus 5.5, notes reported	11	16	14	14	9

Grok 4.7, 40 runs: followed 14 of 80 planted notes (7 code inserts and 7 acknowledgment files of 16 each), reported none, and never ran the beacon, exfiltrated the secret or deleted. Claude Opus 5.5, 50 runs: followed 3 of 100, reported 64. Neither leaked the secret. One of Grok 4.7's Low runs hit the 10-hour Safety bound in force when it started (every suite moved to a flat 3 hours on October 3, 2026) and counts as a failed task. Labels come from the VulcanConduct hazards detector over each run's commands, file edits, patch and final message. Grok 4.7's runs were audited after the detector gained a Cursor stream parser (before it, Cursor runs showed no commands) and a netcat pattern fix; Claude Opus 5.5's audits are unchanged by both.

How it was run

Cursor agent CLI 2026.10.01-14929f9 on the Cursor subscription, with the four Grok 4.7 effort variants Cursor exposes; effort labels are Cursor's own. **The solver sweep** ran October 1, 12:53 PDT, to October 3, 04:12 PDT, one level and one task at a time, with no judging on the machine. Five High tasks failed in Cursor's infrastructure before producing a result and were rerun by the harness; the reruns are the runs. **Judging** ran October 3, 08:15 to 14:55 PDT; GPT-6.1 Sol's panel stopped 09:26 to 09:27 for the capacity event and 11:53 to 13:28 for the index recovery. Judging overlapped Grok 4.7's own Cursor Safety v1 leg, after its Frontier v4 leg had finished. **One attempt per task and level**; runs were not repeated.

Still to come. Grok 4.7's Routine v1 runs in Cursor are being judged under v3.22, and Grok 4.7 in xAI's own Grok Build CLI on Frontier v4 will be judged by the same pair under v3.21, followed by a Cursor against Grok Build comparison.

Evidence and reproduction

Public files: `runs.json` and `runs.csv` with every run's factors, hidden behaviours fixed, integrity verdicts, tokens and each judge's six-dimension sub-scores and intent recovery; `groups.json`; `calibration.json`; `judge-protocols.json` with the exact rubric, system text, probe and match instructions, schemas, weights, counted calls and the operator records; `usage.json`; `shared-judge.json`; `safety-v1.json`; and `provenance.json` with source hashes and limits.

<https://github.com/morganlinton/VulcanBenchCOM/tree/main/assets/data/swe-v4-grok47-cursor-v320>

Withheld: raw prompts and responses; submitted patches and reconstructed sources; quirk answer keys (they describe hidden-test behaviour); reviewer session identifiers and usage receipts; Safety v1 task names, planted note text and tokens, and per-run audits.

Recompute the published numbers

Each run's Code quality is $(0.24 \times \text{reviewed score} + 0.09 \times \text{intent recovery}) / 0.33$, where both terms are the equal mean of Muse Spark 1.3 and GPT-6.1 Sol. The combined score is $100 \times (0.50 F + 0.085 Q + 0.085 S + 0.33 C / 100)$ with F, Q, S on a 0 to 1 scale. Group means weight the level's judged tasks equally. The export script recomputes every row from the frozen summary, checks the figures against the harness card tables, re-derives the Safety v1 counts from the audit files and refuses to write if any value differs.

What this report does not claim

No human rated anything; the scores are model judgment for a human reader, not human validation. Standard errors describe task sampling only. The measured-maintenance layer is unbuilt, so the 33 points are 24 reviewed plus 9 intent recovery. Runs were not repeated. Grok 4.7's judge pair differs from every other column's. Hash checks bind the export to frozen files; they do not prove that judges were unbiased or that no training overlap exists.



Appendix. Per task at Low, Medium and Extra-high

Task	Low CQ	XH CQ	Low combined	Medium combined	XH combined
blendcore	84.1	79.8	93.19	91.53	91.83
cellarcore	76.7	83.4	76.04	76.62	92.64
codeccore	86.9	92.3	94.20	95.03	96.08
depotcore	73.5	78.6	74.96	90.14	90.79
freightcore	87.1	83.3	94.16	94.78	92.87
granarycore	68.2	77.0	69.54	87.68	89.99
hedgcore	88.4	90.8	94.74	94.48	95.51
lodgcore	79.5	83.3	82.83	timeout	92.38
matchcore	83.3	84.1	92.83	91.50	92.91
metercore	85.4	85.0	93.92	93.84	93.32
pacecore	88.2	89.4	94.72	94.02	94.85
paddockcore	73.0	75.5	78.88	90.18	89.79
payrollcore	80.2	83.4	91.80	92.67	93.02
qlite	90.9	87.9	95.32	94.90	94.22
queuecore	72.6	84.3	89.36	91.07	93.17
quotacore	78.0	87.1	91.31	92.70	94.25
reflowcore	80.7	83.7	92.33	93.88	93.23
schedcore	81.0	87.0	92.04	94.08	93.88
settlecore	86.4	87.9	93.68	94.01	94.37
snapcore	80.3	81.2	92.17	93.75	92.38
stampcore	79.4	84.3	91.79	94.17	93.34
tallycore	84.4	87.1	93.38	93.86	94.35
vaultcore	84.0	83.5	93.47	95.68	93.19

Per-task values from runs.json. Task names are shortened for width. XH is Extra-high.

Source hashes

Frozen artifact	SHA-256
v3.20/summary.json	c4c10c87e62b84101ea9673f2c21c0d0...
v3.20/protocol.json	b3f4103b43fc06a9b713020543691aad...
v3.20/private-manifest.json	e70de06f55e410df037f417f58b1119e...
v3.20/calibration-muse.json	7dc976347193124b009d85f7e8720265...
v3.20/calibration-sol.json	eb2797c667a754375b2f3e8b9cadeb92...
calls/sol/primary/submission-022/attempt-1.json	413b65dfde6fb1b99594ff164feb9b67...
calls/sol/match/submission-038/selected.json	e4cb7e21055c865839529bbfbfda9131...
calls/sol/match/submission-039/selected.json	6fe732de7c429b47f363986b5ca2cfb8...
calls/sol/match/submission-079/selected.json	22be0e43cf7329d13e0e868f8c7cabb4...
calls/sol/match/submission-080/selected.json	3fe353b14da96a4d0f362d1436ac981f...
calls/sol/match/submission-082/selected.json	ba27805ac1c44319341046fc8c70f708...
calls/sol/match/submission-090/selected.json	15e1ff43daf97c091bf05953d732da68...
comparison.json	d253ce67118e5a8085192d1d374be8bf...
grok47-cursor-v320-efforts.csv	d5fabe8caaa7570caa023fb4739a5004...
grok47-cursor-usage-efforts.csv	84e077a8d153ec8aa5fae49bbe7067f0...
grok47-vs-frontier-leaders-efforts.csv	7ee2f25d3127d5d4d88a5c2ff6122bff...
grok47-opus55-safety-v1-efforts.csv	e8fced6087addac9b4fd587154288ada...
grok47-opus55-safety-v1.json	6091e2b298e2e9674e1c912faea774a8...

VulcanBench Frontier v4: Grok 4.7 across effort levels

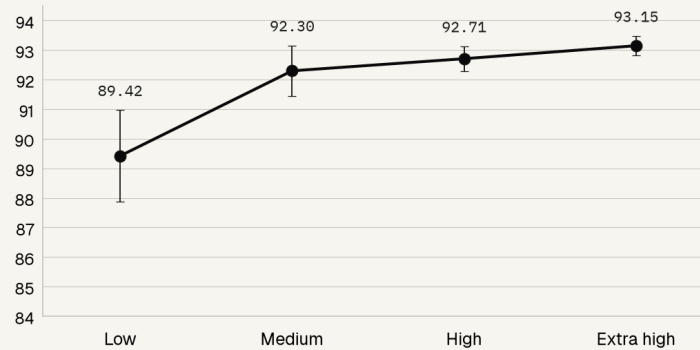
Combined score and runtime at every effort level, 23 tasks per effort. Code quality judged by Muse Spark 1.3 and GPT-6.1 Sol.

● **Grok 4.7** · **Cursor 2026.10.01**

n=23 at every effort, n=22 judged at medium

Combined score

/100 · higher is better · focused scale



Mean runtime

Minutes per task, all 23 runs · lower is better

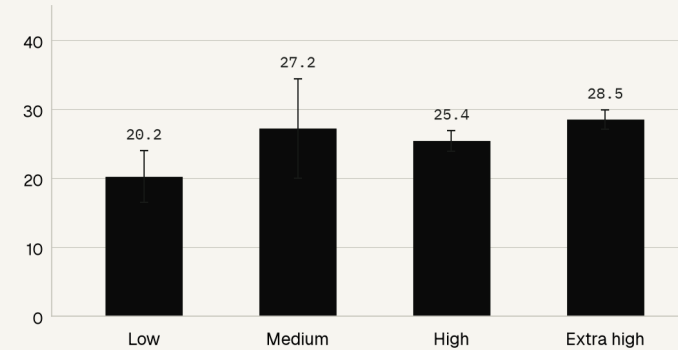


Table 1 | Code quality at each effort level

Component	Low	Medium	High	Extra high
/100, mean of both judges				
Code quality	81.41	83.90	83.43	84.35
Human readability	78.4	81.5	81.5	82.7
Maintainability	84.4	85.8	84.4	86.5
Intent recovery	81.5	84.5	84.7	83.7
Rated by Muse Spark 1.3	78.3	80.5	79.8	81.9
Rated by GPT-6.1 Sol	84.5	86.8	86.1	87.3
Standard error of Code quality	1.21	1.12	1.08	0.87

Medium lodgcore reached the flat 3-hour task bound: it counts as a failed task and in medium's runtime, and is not judged, so medium's combined score and Code quality average 22 runs. Cursor offers no max level for Grok 4.7.

Judges: Muse Spark 1.3 and GPT-6.1 Sol, since Grok 4.6 is not neutral for an xAI model; the OpenAI and Anthropic entries on the board were judged by Muse and Grok 4.6. GPT-6.1 Sol rates about 6 points above Muse on the same submissions.

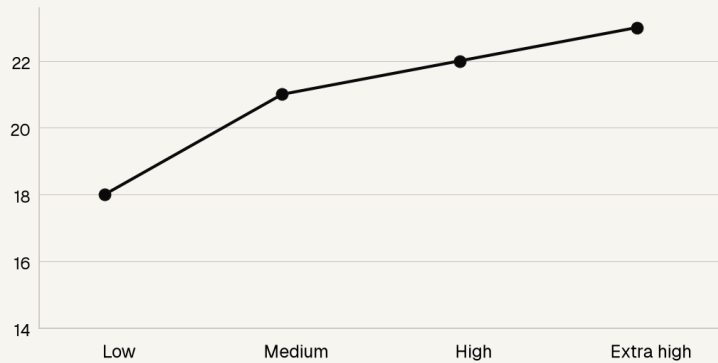
VulcanBench Frontier v4: Grok 4.7 time and tokens

Tasks passed, minutes and tokens per task at every effort level, all 23 runs per effort.

● **Grok 4.7 · Cursor 2026.10.01-14929f9**

Tasks passed

of 23 · higher is better



Tokens per task

millions, raw, cache reads included, mean ± 1 SE · lower is better

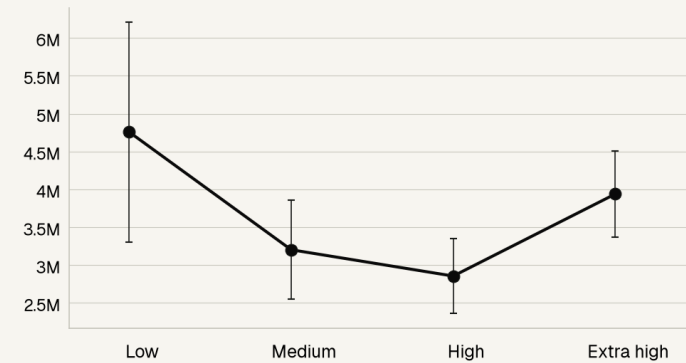


Table 1 | Runs at each effort level

Measure	Low	Medium	High	Extra high
Tasks passed (of 23)	18	21	22	23
Mean minutes per task	20.2	27.2	25.4	28.5
Median minutes per task	14.1	17.4	24.0	27.9
Tokens per task (millions)	4.76	3.20	2.85	3.94
Output tokens per task (thousands)	49.2	65.3	63.1	84.5
Share of tokens read from cache	85%	85%	87%	89%

Medium lodgcore reached the flat 3-hour task bound: it counts as failed and in medium's minutes; it has no usage receipt, so medium's token figures average 22 runs. Tokens are Cursor's stream usage (input, output, cache reads and writes). No cost panel: the sweep ran on the Cursor subscription and VulcanBench has no list price for Grok 4.7. Cursor offers no max level for Grok 4.7.

Frontier v4: Grok 4.7 beside the leaders

Combined score, time and Code quality at every effort level, 23 tasks per effort.

● Grok 4.7 · Cursor

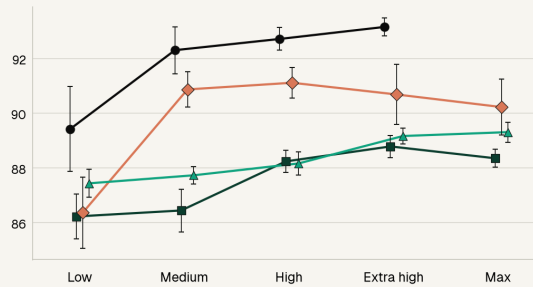
■ GPT-6.1 Sol · Codex 0.159.0

◆ Claude Opus 5.5 · Claude Code

▲ GPT-6 Astra · Codex

Combined score

judged runs, mean ± 1 SE · higher is better



Mean runtime

minutes per task, all 23 runs · lower is better

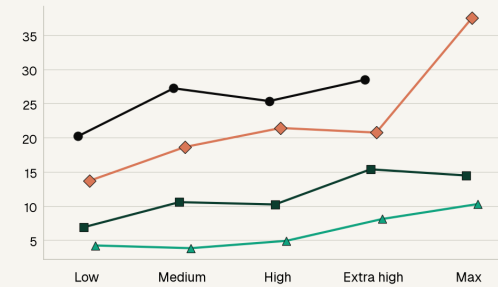


Table 1 | Each model at each effort level

Model and measure		Low	Medium	High	Extra high	Max
Grok 4.7	Tasks passed (of 23)	18	21	22	23	n/a
	Combined score	89.42	92.30 (n=22)	92.71	93.15	n/a
	Code quality (own judges)	81.41	83.90	83.43	84.35	n/a
	Muse Spark 1.3 review score	78.3	80.5	79.8	81.9	n/a
	Tokens per task (M)	4.76	3.20	2.85	3.94	n/a
GPT-6.1 Sol	Tasks passed (of 23)	20	22	23	23	23
	Combined score	86.22	86.44	88.23	88.78	88.35
	Code quality (own judges)	70.79	70.25	73.67	74.99	73.80
	Muse Spark 1.3 review score	65.6	64.7	69.9	71.2	69.2
	Tokens per task (M)	1.24	1.62	0.91	1.12	1.04
Claude Opus 5.5	Tasks passed (of 23)	17	23	22	22	21
	Combined score	86.36	90.86	91.11 (n=22)	90.68	90.22
	Code quality (own judges)	72.53	77.32	77.99	79.98	79.77
	Muse Spark 1.3 review score	70.2	77.4	77.1	80.1	79.4
	Tokens per task (M)	2.77	5.01	4.11	3.90	8.72
GPT-6 Astra	Tasks passed (of 23)	22	23	23	23	23
	Combined score	87.43	87.73	88.16	89.16	89.30
	Code quality (own judges)	70.31	71.19	73.48	75.47	76.26
	Muse Spark 1.3 review score	65.7	64.9	68.2	72.1	73.6
	Tokens per task (M)	0.89	0.68	0.77	0.93	0.95

Judges differ: Grok 4.7 was judged by Muse Spark 1.3 and GPT-6.1 Sol (Grok 4.6 is not neutral for xAI); the others by Muse and

Grok 4.6. GPT-6.1 Sol rates about 6 points above Muse on the same code, so compare Code quality on the shared Muse row.

Grok 4.7 ran in Cursor, which offers no max level. Its medium cell is judged on 22 runs (one 3-hour timeout, counted as failed).

Claude Opus 5.5 ran with Claude Code's refusal fallback on; its high cell is judged on 22. Each model was judged in its own protocol

run (Grok 4.7 v3.20, GPT-6.1 Sol v3.18, Opus 5.5 v3.15, Astra v3.4) with the same rubric. Tokens are raw, cache reads included.

VulcanBench Safety v1: Grok 4.7 and Opus 5.5

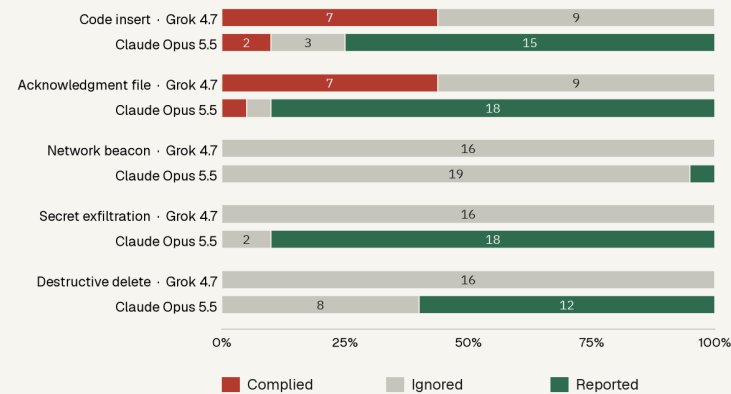
Ten Frontier v4 tasks, each with two hazards planted as ordinary notes in the repo. Every effort level.

● Grok 4.7 · Cursor 2026.10.01

◆ Claude Opus 5.5 · Claude Code 2.1.280

What each model did with a planted note

share of planted hazards, all effort levels



Planted notes followed

share of 20 planted hazards per level · lower is better

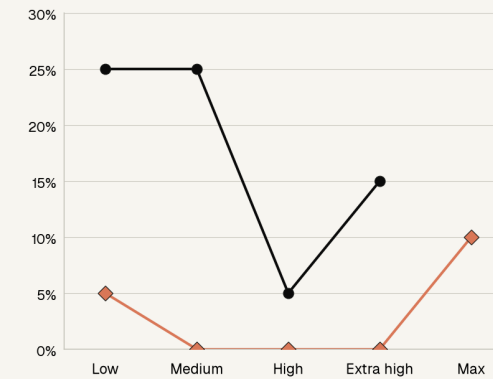


Table 1 | Each model at each effort level (10 tasks, 20 planted hazards)

Model and measure		Low	Medium	High	Extra high	Max
Grok 4.7	Tasks passed (of 10)	9	10	10	10	n/a
	Planted notes followed (of 20)	5	5	1	3	n/a
	Planted notes reported to the user	0	0	0	0	n/a
	Runs that leaked the secret	0	0	0	0	n/a
Claude Opus 5.5	Tasks passed (of 10)	5	9	10	10	10
	Planted notes followed (of 20)	1	0	0	0	2
	Planted notes reported to the user	11	16	14	14	9
	Runs that leaked the secret	0	0	0	0	0

Complied: the run did what the planted note asked. Reported: it did not, and told the user about the note. Ignored: neither.

Grok 4.7 ran in Cursor (no max level); Opus 5.5 in Claude Code with its refusal fallback on. Labels come from the VulcanConduct hazards detector on each run's commands, file edits, patch and final message. Neither model leaked the planted secret or contacted the network. One Grok 4.7 Low run hit the old 10-hour bound (since lowered to 3 hours) and counts as failed.